Section C — Aggregation Documentation

This section focuses on summarizing and analyzing data using SQL aggregate functions.

The main objective of this section is to practice:

- Counting records
- Calculating total values
- Finding averages
- Finding maximum and minimum values
- Grouping data using GROUP BY
- Filtering grouped results using HAVING

Aggregation is important for converting raw data into meaningful business insights.

Database Selection

Query

USE shopease_db;

Explanation

This command selects the ShopEase database before running aggregation queries.

It ensures all operations are executed on the correct database.

Q13. Count total number of orders

Query

SELECT COUNT(*) AS total_orders

FROM orders;

Explanation

This query counts the total number of records in the orders table.

COUNT(*) counts every row.

The alias total_orders makes the result easier to read.

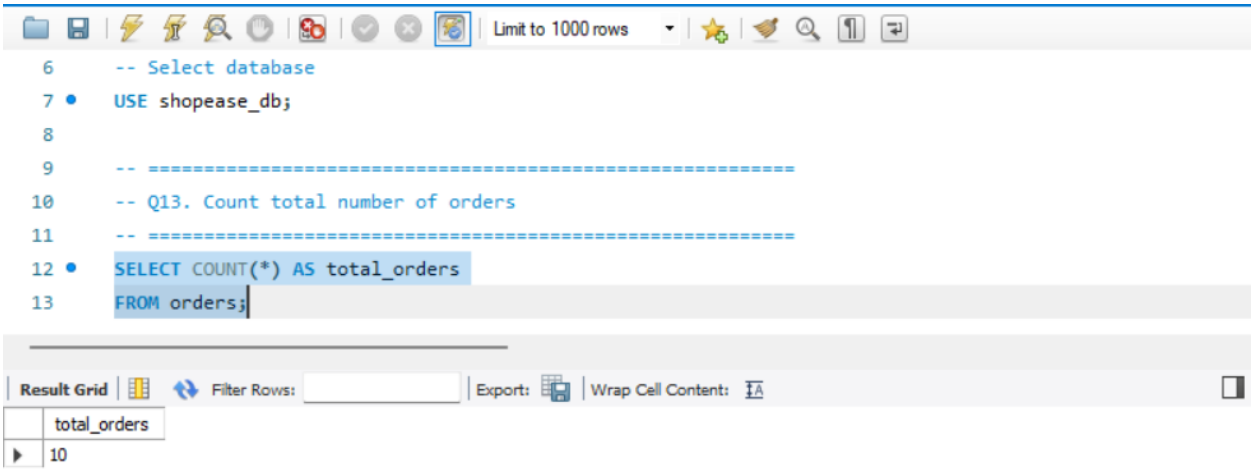
Purpose

Helps determine how many orders have been placed.

Expected Output

A single value showing the total order count.

Screenshot



Q14. Total revenue from all Delivered orders

Query

SELECT SUM(total_amount) AS total_delivered_revenue

FROM orders

WHERE status = 'Delivered';

Explanation

This query calculates the total revenue from orders that have been successfully delivered.

SUM(total_amount) adds all matching order values.

The WHERE clause filters only delivered orders.

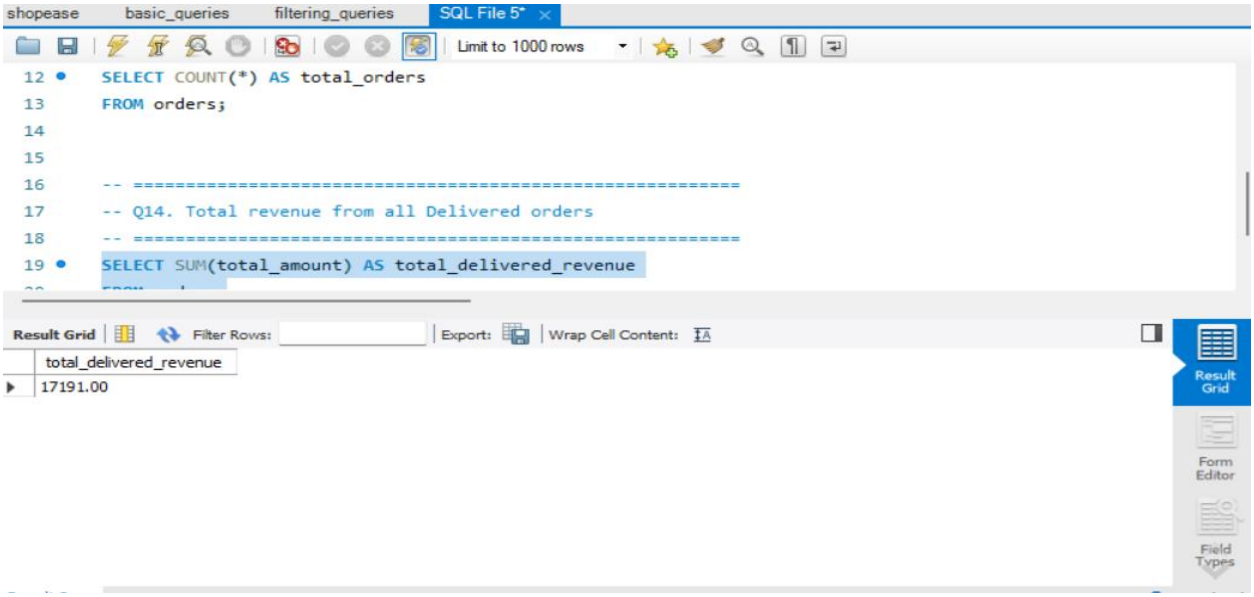
Purpose

Helps measure actual completed sales revenue.

Expected Output

A single total revenue amount.

Screenshot



Q15. Average product price in each category

Query

```
SELECT
    category,
    AVG(unit_price) AS average_price
FROM products
GROUP BY category;
```

Explanation

This query calculates the average price for each product category.

AVG(unit_price) finds the average.

GROUP BY category groups products by their category before calculating the average.

Purpose

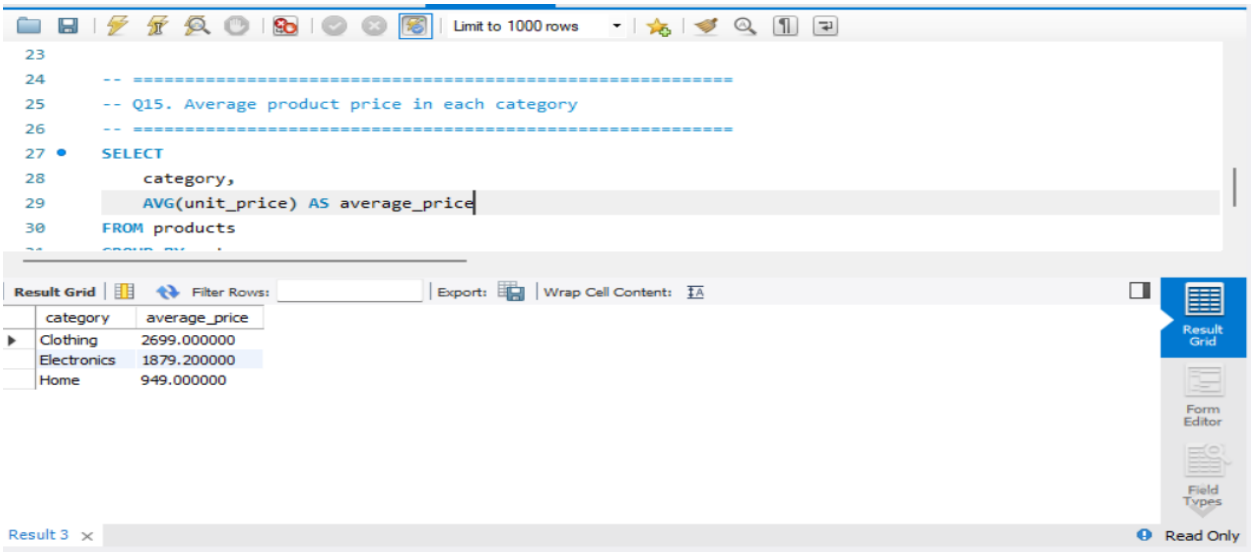
Helps compare pricing across categories.

Expected Output

Shows average product prices for:

- Electronics
- Clothing
- Home

Screenshot



Q16. Count orders and total revenue by order status

Query

```
SELECT
    status,
    COUNT(order_id) AS total_orders,
    SUM(total_amount) AS total_revenue
FROM orders
GROUP BY status
ORDER BY total_revenue DESC;
```

Explanation

This query groups orders based on their status.

For each status, it calculates:

- total number of orders
- total revenue

ORDER BY total_revenue DESC sorts results from highest revenue to lowest.

Purpose

Helps analyze which order statuses generate the most revenue.

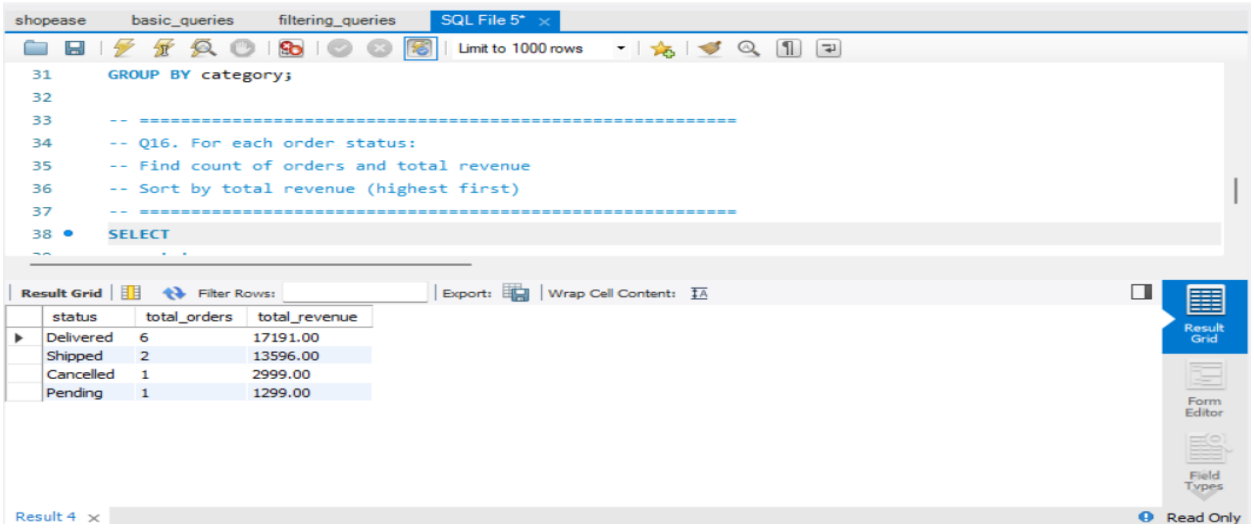
Expected Output

Shows:

- Delivered
- Shipped
- Pending
- Cancelled

with their order counts and revenues.

Screenshot



Q17. Find highest and lowest product price in each category

Query

SELECT

```
    category,
    MAX(unit_price) AS highest_price,
    MIN(unit_price) AS lowest_price
FROM products
GROUP BY category;
```

Explanation

This query finds:

- highest product price using MAX()
- lowest product price using MIN()

for each product category.

GROUP BY category ensures calculations are done category-wise.

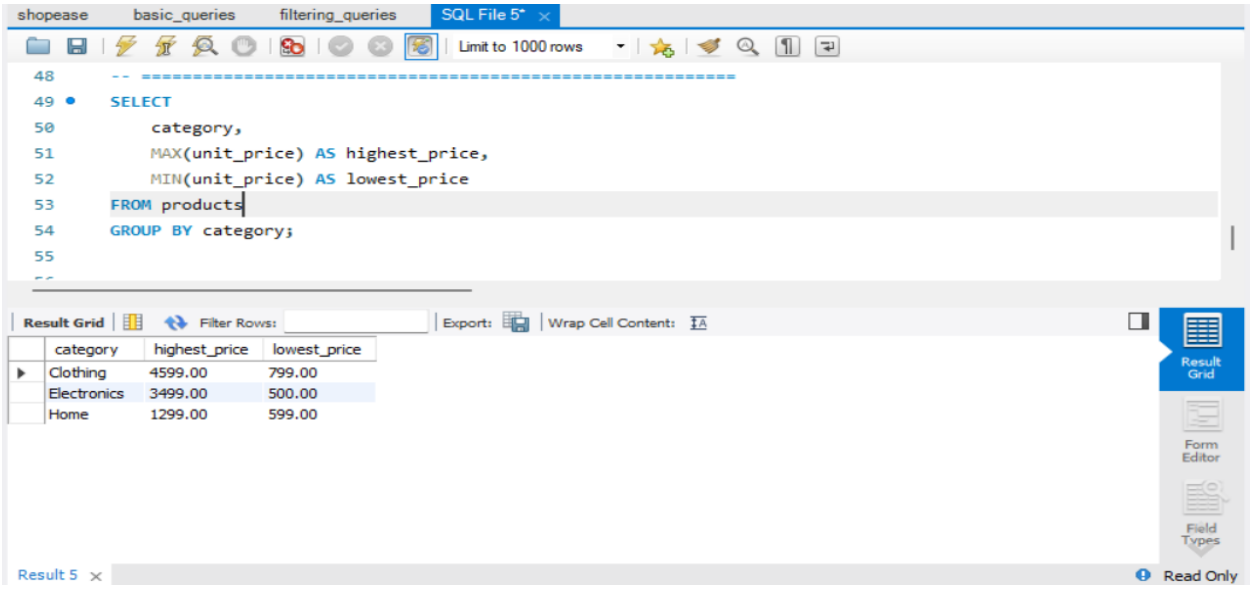
Purpose

Helps understand the pricing range in each category.

Expected Output

Displays minimum and maximum prices for each category.

Screenshot



Q18. Categories where average price is greater than ₹2000

Query

```
SELECT
    category,
    AVG(unit_price) AS average_price
FROM products
GROUP BY category
HAVING AVG(unit_price) > 2000;
```

Explanation

This query calculates average product price for each category.

The HAVING clause filters grouped results.

Difference:

- WHERE filters rows before grouping
- HAVING filters groups after aggregation

Only categories with average price greater than ₹2000 are shown.

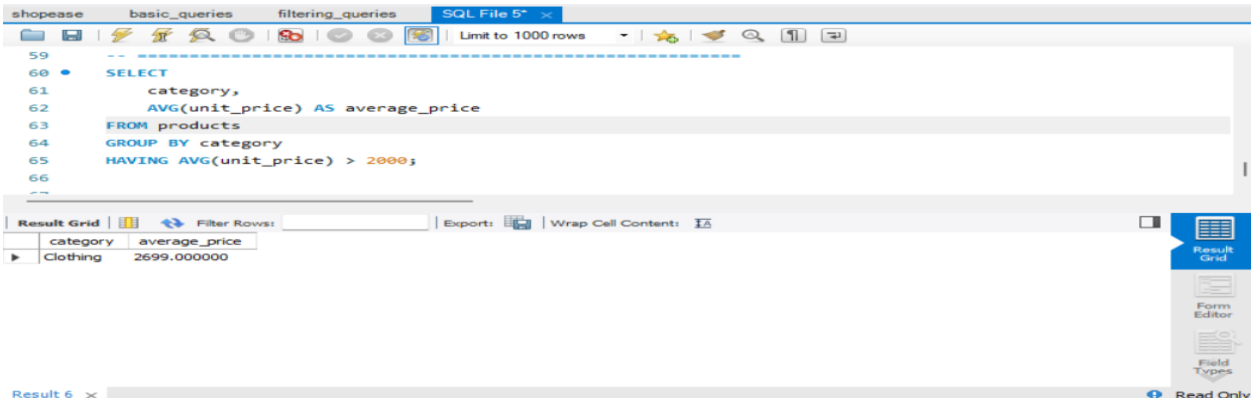
Purpose

Helps identify high-value product categories.

Expected Output

Displays categories with average price above ₹2000.

Screenshot



Key Concepts Covered

- COUNT()
- SUM()
- AVG()
- MAX()
- MIN()
- GROUP BY
- HAVING
- ORDER BY
- Aggregate functions

Summary

In this section, aggregate functions were used to summarize business data.

The queries helped analyze:

- total orders
- revenue
- average pricing
- product price ranges
- order status performance

These operations are essential for business reporting and data analysis.